1. Platform Overview

SYTU is a subscription-based professional networking and portfolio platform targeting students, developers, designers, freelancers, and entrepreneurs in India. It combines real-time stranger discovery, portfolio building, GitHub integration, and a collaboration hub into one ecosystem at ₹49/month.

This document covers the complete system design — architecture decisions, data flow, stack choices, MongoDB schemas, REST + WebSocket API definitions, and the full project folder structure for both frontend and backend.

2. Technology Stack

SYTU uses the MERN stack (MongoDB, Express.js, React via Next.js, Node.js) as the core, extended with Redis, BullMQ, and Socket.IO where needed. Every choice below is open-source and free to self-host.

2.1 Full Stack Table

Layer	Technology	Why This Choice
Frontend	Next.js 14 (App Router) + TypeScript	SSR for public portfolio SEO, client-side SPA for the dashboard, ISR for discovery feeds
UI Styling	Tailwind CSS	Utility-first, zero runtime, consistent design tokens, PurgeCSS eliminates dead styles
Global State	Zustand	Tiny, no boilerplate, stores JWT in memory (not localStorage) to prevent XSS theft
Server State	TanStack Query (React Query)	Cache + refetch + optimistic updates for API data, reduces redundant network calls
WS Client	Socket.IO Client	Mirrors the server library; handles reconnection, namespaces, and rooms automatically
Backend Runtime	Node.js 20 LTS + Express.js	Non-blocking I/O ideal for I/O-bound workloads (DB reads, S3, GitHub API calls)
Database	MongoDB via Mongoose	Flexible BSON documents fit user profiles, JSONB-like portfolio sections, and project arrays naturally
Cache	Redis (open-source)	Session store, API cache, Socket.IO pub/sub adapter, BullMQ job queue backing
Job Queue	BullMQ (backed by Redis)	Background tasks: email, PDF generation, GitHub sync. No extra infrastructure needed.

Real-Time	Socket.IO 4.x + Redis adapter	WebSocket chat, online presence, live notifications across multiple server instances
File Storage	AWS S3 + CloudFront CDN	S3 for durability; CloudFront caches images/assets at edge nodes near Indian users
Email	AWS SES (free tier: 62k/month)	Transactional email — verification, OTP, subscription alerts. No third-party cost at MVP.
Payments	Razorpay	Best Indian payment gateway — UPI, cards, net banking from one integration
Auth	JWT (access + refresh) + Passport.js	Stateless auth scales horizontally; Passport.js simplifies GitHub OAuth to a strategy
Process Manager	PM2	Free, runs Node.js in cluster mode (all CPU cores), auto-restarts on crash
Reverse Proxy	Nginx	Free, sits in front of Node.js — SSL termination, static file serving, load balancing across PM2 instances
Containerisation	Docker + Docker Compose (dev)	Reproducible local dev environment for MongoDB, Redis, and the API
CI/CD	GitHub Actions	Free for public repos, sufficient for private at MVP; build → test → deploy on merge
Cloud	AWS EC2 (t3.medium) or Railway.app	EC2 for full control; Railway as a cheaper managed alternative at early stage

2.2 When Redis and BullMQ Are Used

Redis is not added for complexity; it solves three concrete problems that would otherwise require paid services or architectural rewrites:

- **API caching:** Discovery feed computation is expensive (skill + interest overlap scoring across thousands of users). Redis caches the result per user for 5 minutes. Without this, every feed refresh hits MongoDB with a complex aggregation.
- **Socket.IO scaling:** When the app runs on multiple Node.js processes (via PM2 cluster) or multiple servers, a WebSocket message sent to Process A cannot reach a user connected to Process B. Redis pub/sub (via @socket.io/redis-adapter) acts as the shared broadcast channel between processes — zero cost, zero external dependency.
- **Background jobs:** Email sending, PDF generation, and GitHub sync must not block the HTTP response. BullMQ uses Redis sorted sets to queue jobs with priority, retries, and delay. No Kafka or RabbitMQ needed at this scale.

3. System Architecture

3.1 Architecture Layers

Layer	Components and Responsibility
Client	Next.js web app. Renders UI, manages client state, calls REST API and Socket.IO server.
Edge / CDN	CloudFront serves Next.js static assets and S3 media files from edge locations near users.
Reverse Proxy	Nginx on the server — terminates TLS, serves static files, load-balances to PM2 Node.js cluster workers.
API Layer	Express.js app. Single entry point: validates JWT, enforces rate limits, routes to service modules.
Real-Time Layer	Socket.IO server (separate process). Handles WebSocket connections for chat, presence, notifications.
Service Layer	Business logic modules: Auth, User, Portfolio, Discovery, Chat, Projects, Collab, Billing, Notifications.
Cache Layer	Redis: API cache with TTL, Socket.IO pub/sub adapter, BullMQ job queues, JWT blocklist.
Database Layer	MongoDB (via Mongoose): all persistent data. Collections per domain entity.
File Storage	AWS S3: profile pictures, project screenshots, chat attachments, PDF resumes.
Job Workers	BullMQ workers: email (AWS SES), PDF (Puppeteer), GitHub sync, discovery precompute.
External APIs	GitHub OAuth + REST API, Razorpay payments.

3.2 Full Request Lifecycle

Walkthrough of a typical authenticated request — user loading their discovery feed:

- Browser requests sytu.com. CloudFront serves the pre-built Next.js HTML/CSS/JS from the nearest edge node. Subsequent navigation is handled client-side.
- User action triggers GET /api/v1/discovery/feed. Request includes Authorization: Bearer <access_token> header.
- Nginx receives the HTTPS request, terminates TLS using the Let's Encrypt certificate, forwards plain HTTP to one of the PM2 cluster workers on localhost:5000.
- Express auth middleware validates the JWT signature and expiry. Checks the token JTI against the Redis blocklist (for logged-out tokens). Returns 401 if invalid.
- Rate limit middleware checks request count for this user in Redis. Blocks at 100 requests/minute with a 429 response.
- Router forwards to the Discovery service. Service checks Redis for a cached feed (key: feed:<userId>, TTL: 5 min). Cache HIT → return immediately, no DB query.

- Cache MISS: Discovery service runs a MongoDB aggregation — lookup users whose skills array intersects with the requester's skills, score by overlap, exclude already-connected users, limit 20.
- Result is written to Redis with TTL, returned as JSON. Total API time target: < 500 ms.
- Next.js hydrates the page and renders the feed cards.

3.3 Real-Time Message Lifecycle

- User A types a message. Socket.IO client emits chat:send with {conversationId, content}.
- Socket.IO server receives the event. Saves message document to MongoDB messages collection.
- Server emits chat:receive to User B's personal room (room ID = userId). If User B is connected to a different PM2 worker, the Redis adapter broadcasts across workers.
- If User B is offline, the Notification service creates a notification document and enqueues an email job in BullMQ.
- BullMQ email worker picks the job, calls AWS SES to send the email.

3.4 File Upload Lifecycle (Presigned S3)

- Frontend requests POST /api/v1/upload/presign with {filename, contentType}.
- Backend generates a presigned S3 URL valid for 5 minutes using the AWS SDK. Returns {presignedUrl, fileKey}.
- Frontend uploads the file directly to S3 using a PUT request to the presigned URL. The API server never touches the file bytes.
- On upload success, frontend calls the relevant API (e.g. PATCH /users/me with {profileImageUrl: fileKey}). Backend stores the S3 key; CloudFront URL is derived at read time.

This keeps large file transfers off the API servers entirely, protecting their memory and CPU.

4. Security Design

Threat / Requirement	Implementation
Password storage	bcrypt with cost factor 12. Passwords are never stored in plain text or retrievable.
Authentication tokens	Short-lived access JWT (15 min). Long-lived refresh token (7 days) stored as httpOnly, SameSite=Strict, Secure cookie. Access token stored in memory (not localStorage).
Token revocation	JWT JTI (token ID) added to Redis blocklist on logout with TTL matching remaining token lifetime.
SQL/NoSQL injection	Mongoose uses parameterized queries exclusively. User input is never interpolated into query strings.
XSS	React escapes all output by default. Content-Security-Policy header set by Nginx. Input sanitised with DOMPurify on frontend.
CSRF	SameSite=Strict on all cookies. CSRF token double-submit for state-changing requests.
Rate limiting	express-rate-limit backed by Redis. Login: 5 attempts per 15 min per IP. API: 100 requests per min per user.
Transport security	HTTPS everywhere. Nginx terminates TLS with Let's Encrypt certificate (free, auto-renews). HTTP redirects to HTTPS.
File upload safety	MIME type validation on upload. Sharp strips EXIF metadata from images before S3 storage.
Razorpay webhooks	HMAC-SHA256 signature validation on every webhook event before processing.
GitHub token storage	GitHub OAuth access tokens stored AES-256 encrypted in MongoDB.
Admin routes	Separate admin middleware checks role: 'admin' claim in JWT. All admin mutations logged to audit_logs collection.
Environment secrets	All secrets in .env files, never committed to Git. Production secrets in AWS Secrets Manager.

5. MongoDB Schema Design

MongoDB is used for all persistent data. Collections are designed to minimise expensive cross-collection lookups for the most frequent read patterns. Embedding is used for small, bounded sub-documents (skills, education entries); referencing is used for large or independently queried entities (messages, projects).

5.1 users

```
Collection: users
_____________________________________________________

{
  _id:                ObjectId,
  username:           String,   // unique, lowercase, indexed
  email:              String,   // unique, lowercase, indexed
  passwordHash:       String,   // bcrypt. null for OAuth-only accounts
  name:               String,
  bio:                String,   // max 300 chars
  profileImageUrl:    String,   // S3 key
  headline:           String,   // e.g. 'Full-Stack Developer at XYZ'
  linkedinUrl:        String,
  websiteUrl:         String,

  // embedded arrays — small, bounded, loaded with profile
  skills: [{
    name:        String,         // lowercase
    proficiency: String,         // 'beginner' | 'intermediate' | 'expert'
  }],
  interests: [String],           // lowercase tags

  // flags
  isPremium:          Boolean,  // denormalised from subscriptions for fast auth checks
  isEmailVerified:    Boolean,
  isSuspended:        Boolean,
  role:               String,   // 'user' | 'admin'

  // online presence
  lastSeen:           Date,     // updated on every API request

  // github
  githubId:           String,   // unique sparse index
  githubUsername:     String,
```

```
  createdAt:           Date,
  updatedAt:           Date,
}

Indexes: { username: 1 } unique, { email: 1 } unique, { 'skills.name': 1 },
         { interests: 1 }, { githubId: 1 } sparse unique
```

5.2 connections

```
Collection: connections
_____________________________________________________

{
  _id:         ObjectId,
  senderId:    ObjectId,   // ref: users
  receiverId:  ObjectId,   // ref: users
  status:      String,     // 'pending' | 'accepted' | 'rejected' | 'blocked'
  createdAt:   Date,
  updatedAt:   Date,
}

Indexes: { senderId: 1, receiverId: 1 } unique compound,
         { receiverId: 1, status: 1 }  // for inbox queries
```

5.3 conversations + messages

```
Collection: conversations
_____________________________________________________

{
  _id:           ObjectId,
  participants:  [ObjectId],  // exactly 2 user IDs for 1-to-1 chat
  lastMessage: {
    content:   String,
    senderId:  ObjectId,
    sentAt:    Date,
  },
  createdAt:     Date,
}

Indexes: { participants: 1 }  // find conversations by user ID
```

```
──────────────────────────────────────────────────

Collection: messages

──────────────────────────────────────────────────

{
  _id:              ObjectId,
  conversationId:   ObjectId,   // ref: conversations, indexed
  senderId:         ObjectId,   // ref: users
  content:          String,     // null if file message
  fileUrl:          String,     // S3 key. null if text message
  fileType:         String,     // 'image' | 'pdf' | 'file'
  isRead:           Boolean,
  createdAt:        Date,       // indexed for cursor pagination
}

Indexes: { conversationId: 1, createdAt: -1 }
```

5.4 projects

```
Collection: projects

──────────────────────────────────────────────────

{
  _id:            ObjectId,
  userId:         ObjectId,   // ref: users, indexed
  title:          String,
  description:    String,
  category:       String,     // 'Web' | 'Mobile' | 'ML' | 'Design' | 'Other'
  techStack:      [String],   // ['React', 'Node.js', ...]
  githubUrl:      String,
  demoUrl:        String,
  screenshotUrls: [String],   // up to 5 S3 keys
  teamSize:       Number,
  status:         String,     // 'in_progress' | 'completed' | 'abandoned'
  isFeatured:     Boolean,    // pinned to portfolio homepage
  createdAt:      Date,
  updatedAt:      Date,
}

Indexes: { userId: 1 }, { techStack: 1 }, { category: 1 }
```

5.5 portfolios

```
Collection: portfolios
──────────────────────────────────────────────────

{
  _id:        ObjectId,
  userId:     ObjectId,   // ref: users, unique index
  about:      String,
  headline:   String,

  experience: [{
    company:     String,
    role:        String,
    from:        Date,
    to:          Date,     // null = current
    description: String,
  }],

  education: [{
    institution: String,
    degree:      String,
    from:        Date,
    to:          Date,
  }],

  achievements: [{
    title:       String,
    description: String,
    date:        Date,
  }],

  certifications: [{
    name:   String,
    issuer: String,
    url:    String,
    date:   Date,
  }],

  theme:          String,    // 'default' | 'minimal' | 'dark'
  isPublished:    Boolean,
  viewCount:      Number,    // approximate, updated in batches
  updatedAt:      Date,
}
```

```
Indexes: { userId: 1 } unique
```

5.6 subscriptions

```
Collection: subscriptions
──────────────────────────────────────────────────
{
  _id:                    ObjectId,
  userId:                 ObjectId,   // ref: users, unique index
  razorpayOrderId:        String,
  razorpayPaymentId:      String,
  razorpaySubscriptionId: String,
  plan:                   String,     // 'monthly'
  amountPaise:            Number,     // 4900 = ₹49
  status:                 String,     // 'active' | 'cancelled' | 'expired' | 'trial'
  startDate:              Date,
  expiryDate:             Date,       // indexed for expiry cron
  autoRenew:              Boolean,
  createdAt:              Date,
}

Indexes: { userId: 1 } unique, { expiryDate: 1, status: 1 }
```

5.7 collaboration_posts

```
Collection: collaboration_posts
──────────────────────────────────────────────────
{
  _id:          ObjectId,
  userId:       ObjectId,   // ref: users
  type:         String,     // 'developer'|'designer'|'startup'|'freelance'|'hackathon'
  title:        String,
  description:  String,
  skillsNeeded: [String],
  isRemote:     Boolean,
  isOpen:       Boolean,    // closed when team is found
  expiresAt:    Date,       // auto-close after 30 days
  createdAt:    Date,
}
```

```
Indexes: { type: 1 }, { skillsNeeded: 1 }, { expiresAt: 1, isOpen: 1 }
```

5.8 notifications

```
Collection: notifications
──────────────────────────────────────────────────
{
  _id:       ObjectId,
  userId:    ObjectId,   // recipient. ref: users, indexed
  type:      String,     //
'connection_req'|'message'|'portfolio_view'|'sub_expiry'|'collab_req'
  title:     String,
  body:      String,
  actionUrl: String,     // deep link within app
  isRead:    Boolean,
  createdAt: Date,
}

Indexes: { userId: 1, createdAt: -1 }, { userId: 1, isRead: 1 }
```

5.9 github_integrations

```
Collection: github_integrations
──────────────────────────────────────────────────
{
  _id:              ObjectId,
  userId:           ObjectId,   // ref: users, unique
  accessTokenEnc:   String,     // AES-256 encrypted GitHub OAuth token
  githubUserId:     String,
  reposData:        Mixed,      // cached repo list from GitHub API
  contributionData: Mixed,      // cached contribution graph
  languagesData:    Mixed,      // aggregated language breakdown
  lastSyncedAt:     Date,
}

Indexes: { userId: 1 } unique
```

5.10 reports + audit_logs

```
Collection: reports
{
```

```
  _id:           ObjectId,
  reporterId:    ObjectId,  // who filed it
  targetUserId:  ObjectId,  // who was reported
  reason:        String,    // 'spam'|'fake_profile'|'harassment'|'inappropriate'
  description:   String,
  status:        String,    // 'open' | 'resolved' | 'dismissed'
  adminNote:     String,
  createdAt:     Date,
}

Collection: audit_logs
{
  _id:        ObjectId,
  adminId:    ObjectId,
  action:     String,      // 'suspend_user', 'ban_user', 'resolve_report', ...
  targetId:   ObjectId,
  meta:       Mixed,       // additional context
  createdAt:  Date,
}
```

6. REST API Reference

Base URL: https://api.sytu.com/v1. All endpoints return JSON. Authentication via Authorization: Bearer <access_token> unless marked Public.

Standard error response: { success: false, code: 'ERROR_CODE', message: 'Human readable message' }

6.1 Authentication

Method + Path	Auth	Description
POST /auth/register	Public	Body: {name, email, password, mobile}. Creates account, sends verification email via BullMQ job.
POST /auth/login	Public	Body: {email, password}. Returns {accessToken}. Sets httpOnly refresh cookie.
POST /auth/logout	User	Adds access token JTI to Redis blocklist. Clears refresh cookie.
POST /auth/refresh	Cookie	Reads httpOnly refresh cookie, validates it, returns new {accessToken}.
POST /auth/verify-email	Public	Body: {token}. Validates signed JWT from email link. Sets isEmailVerified=true.
POST /auth/send-otp	Public	Body: {mobile}. Sends 6-digit OTP via SMS (2Factor.in free tier).
POST /auth/verify-otp	Public	Body: {mobile, otp}. Validates OTP stored in Redis (TTL: 10 min).
POST /auth/forgot-password	Public	Body: {email}. Sends reset link with signed JWT (expiry: 1 hr).
POST /auth/reset-password	Public	Body: {token, newPassword}. Validates token, updates passwordHash.
GET /auth/github	Public	Initiates GitHub OAuth — Passport.js redirects to GitHub.
GET /auth/github/callback	Public	GitHub OAuth callback. Returns JWT on success. Handles both signup and account-link.

6.2 Users & Profiles

Method + Path	Auth	Description
GET /users/:username	Public	Public profile — name, bio, skills, headline, portfolio link. Cached in Redis for 2 min.
GET /users/me	User	Full authenticated user profile including private fields.
PATCH /users/me	User	Partial update: name, bio, headline, linkedinUrl, websiteUrl.
POST /users/me/avatar	User	Returns presigned S3 URL + fileKey. Frontend uploads directly to S3.
POST /users/me/skills	User	Body: {name, proficiency}. Pushes to skills array.

DELETE /users/me/skills/:name	User	Pulls skill from array by name.
POST /users/me/interests	User	Body: {interest}. Pushes to interests array.
DELETE /users/me/interests/:name	User	Pulls interest from array.
GET /users/search	User	Query: ?q=&skills=&interests=. Full-text search via MongoDB text index on name+username+bio+skills. Advanced filters require Premium.
POST /users/:id/report	User	Body: {reason, description}. Creates report document.

6.3 Discovery & Connections

Method + Path	Auth	Description
GET /discovery/feed	Premium	Paginated recommended users. Served from Redis cache (key: feed:<userId>, TTL: 5 min). Computes match score from skills + interests overlap.
GET /discovery/random	User	One random user not yet connected. Free feature.
POST /connections/request	User	Body: {receiverId}. Creates connection document (status: pending). Emits Socket.IO notification to receiver.
POST /connections/:id/accept	User	Updates status to 'accepted'. Creates conversation document for the pair. Sends notification.
POST /connections/:id/reject	User	Updates status to 'rejected'.
DELETE /connections/:id	User	Removes connection. Does not delete conversation history.
POST /users/:id/block	User	Creates connection doc with status 'blocked'. Blocked user cannot message or connect.
POST /users/:id/follow	User	One-way follow (no request needed). Stored as {followerId, followeeId} in follows collection.
DELETE /users/:id/follow	User	Unfollow.
GET /connections	User	All accepted connections. Includes lastSeen and unread message count.
GET /connections/pending	User	Incoming pending connection requests.

6.4 Chat (REST — initial load + history)

Method + Path	Auth	Description
GET /conversations	User	All conversations sorted by lastMessage.sentAt desc. Includes unread count per conversation.

GET /conversations/:id/messages	User	Paginated messages. Query: ?before=<messageId>&limit=50. Cursor-based pagination using _id.
PATCH /conversations/:id/read	User	Marks all unread messages in conversation as read. Emits chat:read_ack to sender via Socket.IO.
POST /conversations/:id/files	User	Multipart upload — file goes to S3, message document created with fileUrl.

6.5 Portfolio

Method + Path	Auth	Description
GET /portfolio/:username	Public	Public portfolio. ISR in Next.js — revalidates every 60 seconds. Returns 404 if not published.
GET /portfolio/me	User	User's own portfolio (draft or published).
PATCH /portfolio/me	User	Partial update any section: about, headline, experience, education, achievements, certifications, theme.
POST /portfolio/me/publish	User	Sets isPublished=true. Portfolio becomes live at sytu.com/u/:username.
POST /portfolio/me/unpublish	User	Sets isPublished=false.
GET /portfolio/me/pdf	User	Enqueues PDF generation job. Returns {jobId}. Poll GET /portfolio/me/pdf/:jobId for status + download URL.
GET /portfolio/me/analytics	Premium	Returns view counts by day/week, top referrers, approximate geography.

6.6 Projects

Method + Path	Auth	Description
GET /projects	Public	Paginated project showcase. Query: ?category=&tech=&userId=. MongoDB find with index on techStack.
GET /projects/:id	Public	Single project detail.
POST /projects	User	Create project. Body: {title, description, category, techStack, githubUrl, demoUrl, teamSize, status}.
PATCH /projects/:id	User	Update project (own only). Validates ownership in middleware.
DELETE /projects/:id	User	Delete project (own only). Removes screenshot S3 keys.
POST /projects/:id/screenshots	User	Upload up to 5 screenshots. Returns presigned S3 URLs for direct browser upload.
PATCH /projects/:id/feature	User	Toggle isFeatured. Controls pin on portfolio homepage.

6.7 Collaboration Hub

Method + Path	Auth	Description
GET /collaboration	User	Paginated posts. Query: ?type=&skills=&isRemote=. Filters on indexed fields.
POST /collaboration	Premium	Create post. Body: {type, title, description, skillsNeeded, isRemote}.
GET /collaboration/:id	User	Single post detail + creator profile.
PATCH /collaboration/:id/close	User	Own post only. Sets isOpen=false.
POST /collaboration/:id/apply	User	Creates application. Notifies post creator via Socket.IO notification and email.
POST /collaboration/:id/save	User	Bookmarks post. Stored in saves collection {userId, postId}.

6.8 Subscriptions & Payments

Method + Path	Auth	Description
GET /subscription/plans	Public	Returns plan list with pricing. Static, no DB call.
POST /subscription/create	User	Creates Razorpay Order. Returns {orderId, amount, currency, key}.
POST /subscription/verify	User	Body: {orderId, paymentId, signature}. Validates HMAC-SHA256 signature. Creates subscription record. Sets user.isPremium=true.
GET /subscription/status	User	Returns current subscription status and expiryDate.
POST /subscription/cancel	User	Cancels auto-renewal via Razorpay API. Subscription active until expiryDate.
POST /webhooks/razorpay	System	Validates HMAC webhook signature. Handles payment.captured, subscription.halted, subscription.cancelled.

6.9 GitHub Integration

Method + Path	Auth	Description
GET /github/connect	User	Initiates GitHub OAuth for existing user account-linking.
POST /github/disconnect	User	Unlinks GitHub account, deletes stored encrypted token.
GET /github/repos	User	Returns cached repos from github_integrations.reposData. Triggers sync job if lastSyncedAt > 6h.
POST /github/repos/import	User	Body: {repoIds: [...]}. Creates project documents from selected repos.
GET /github/stats	User	Returns contribution graph and language breakdown from cache.

6.10 Notifications

Method + Path	Auth	Description
GET /notifications	User	Paginated notifications. Query: ?unreadOnly=true. Sorted by createdAt desc.
PATCH /notifications/:id/read	User	Mark single notification as read.
PATCH /notifications/read-all	User	Mark all as read.
DELETE /notifications/:id	User	Delete a notification.

6.11 Admin

Method + Path	Auth	Description
GET /admin/users	Admin	Paginated user list. Query: ?q=&role=&isSuspended=&isPremium=.
PATCH /admin/users/:id/suspend	Admin	Set isSuspended=true. Logs to audit_logs. Sends suspension email.
PATCH /admin/users/:id/unsuspend	Admin	Set isSuspended=false. Logs to audit_logs.
DELETE /admin/users/:id	Admin	Permanent account deletion. GDPR. Cascades to remove projects, messages, connections.
GET /admin/reports	Admin	Paginated reports. Query: ?status=open.
PATCH /admin/reports/:id	Admin	Body: {status, adminNote}. Resolve or dismiss. Logs action.
GET /admin/analytics	Admin	Returns: total users, premium users, MRR, new signups by day, retention rate.

7. Real-Time: Socket.IO Event System

Socket.IO server runs as a separate process alongside the REST API. On connection, client sends JWT in handshake.auth.token. Server validates it and joins the socket to the user's personal room (room ID = userId string).

The Redis adapter (@socket.io/redis-adapter) is attached from startup, allowing events to propagate across all PM2 cluster workers without any code changes.

Event Name	Direction	Payload + Behaviour
connect (with auth)	Client → Server	Client sends {auth: {token: accessToken}}. Server validates JWT, joins room(userId), updates lastSeen in Redis, emits presence:online to all connections in Redis.
disconnect	Server event	Server updates lastSeen in MongoDB (via debounced BullMQ job, not inline). Emits presence:offline to connections.
chat:send	Client → Server	Payload: {conversationId, content, fileUrl?}. Server saves message to MongoDB, updates conversation.lastMessage, emits chat:receive to recipient room.
chat:receive	Server → Client	Payload: full message object. Client appends to conversation state.
chat:typing	Client → Server	Payload: {conversationId}. Server re-emits chat:typing to recipient room. Not persisted.
chat:typing_stop	Client → Server	Payload: {conversationId}. Server re-emits to recipient.
chat:read	Client → Server	Payload: {conversationId}. Server marks messages read in MongoDB, emits chat:read_ack to sender.
notification:new	Server → Client	Server pushes notification object when: new connection request, new message (if offline tab), portfolio view milestone, sub expiry warning.
presence:online	Server → Client	Payload: {userId}. Broadcast to all of this user's connections when they connect.
presence:offline	Server → Client	Payload: {userId}. Broadcast to connections on disconnect.

8. Background Job Queues (BullMQ)

All async work is handled via BullMQ workers backed by Redis. Workers are separate processes that can be scaled independently from the HTTP API.

Queue	Jobs and Trigger
email-queue	send_verification_email (on register), send_otp (on request), send_welcome (on verify), send_subscription_confirmation (on payment), send_expiry_warning (3 days before expiryDate, daily cron). All use AWS SES.
pdf-queue	generate_portfolio_pdf — triggered by GET /portfolio/me/pdf. Puppeteer renders the portfolio page HTML, exports to PDF, uploads to S3, stores URL in Redis keyed by jobId for the polling endpoint.
github-sync-queue	sync_github_repos — triggered by GET /github/repos if lastSyncedAt > 6h, or manually via button. Calls GitHub REST API, updates github_integrations document, invalidates Redis cache.
discovery-queue	precompute_discovery_feed — triggered on user login if no cached feed exists. Runs MongoDB aggregation for skill+interest overlap scoring. Stores top-20 results in Redis (TTL: 1hr).
analytics-queue	record_portfolio_view — called on every portfolio page load. Batched writes to MongoDB every 5 minutes to avoid per-request DB writes. Updates portfolio.viewCount in a single bulk operation.
moderation-queue	process_report — on new report creation. Sends admin email alert for reason='harassment'. High-severity reports auto-trigger a soft-suspend pending admin review.
cleanup-queue	expire_subscriptions — daily cron. Finds subscriptions where expiryDate < now AND status='active'. Sets status='expired', sets user.isPremium=false. Enqueues expiry email.

9. Caching Strategy (Redis)

Cache Key Pattern	TTL	What Is Cached + Invalidation
feed:<userId>	1 hour	Precomputed discovery feed (array of 20 user IDs + match scores). Invalidated on skill/interest update.
profile:<username>	2 min	Public profile response. Invalidated on PATCH /users/me.
portfolio:<username>	5 min	Public portfolio response. Invalidated on publish/update.
githubstats:<userId>	6 hours	GitHub repos + contributions response. Invalidated by sync worker.
rate:<ip>:<route>	Per window	Request count per IP per route for rate limiting. Managed by express-rate-limit.
blocklist:<jti>	Remaining token lifetime	Revoked JWT JTI. Checked on every authenticated request.
otp:<mobile>	10 min	6-digit OTP for mobile verification. Single-use (deleted after successful verify).
pdfjob:<jobId>	30 min	PDF generation status and S3 download URL. Polled by frontend.

10. Backend — Project Folder Structure

```
backend/
├── src/
│   ├── app.ts                    Entry point. Creates Express app, registers middleware, mounts routers.
│   ├── server.ts                 Starts HTTP server + Socket.IO server. Connects MongoDB + Redis.
│   │
│   ├── config/                   All configuration and external client initialisation.
│   │   ├── db.ts                 Mongoose connection setup. Retries on failure.
│   │   ├── redis.ts              ioredis client. Single instance shared across app.
│   │   ├── s3.ts                 AWS S3 client + presigned URL helper functions.
│   │   ├── razorpay.ts           Razorpay Node.js SDK initialisation.
│   │   └── env.ts                Reads and validates all environment variables at startup (fail fast).
│   │
│   ├── middleware/               Express middleware — applied globally or per-router.
│   │   ├── auth.ts               Validates JWT from Authorization header. Attaches req.user.
│   │   ├── requirePremium.ts     Checks req.user.isPremium. Returns 403 if false.
│   │   ├── requireAdmin.ts       Checks req.user.role === 'admin'. Returns 403 if false.
│   │   ├── rateLimit.ts          express-rate-limit configs backed by Redis. login, api, upload limits.
│   │   ├── upload.ts             Multer config for in-memory file handling before S3 upload.
│   │   ├── validate.ts           Zod schema validator factory — wraps route handlers with input parsing.
│   │   └── errorHandler.ts       Global Express error handler. Maps errors to status codes and response shape.
│   │
│   ├── modules/                  One subfolder per domain feature.
│   │   │
│   │   ├── auth/
│   │   │   ├── auth.routes.ts    Registers all /auth/* routes. Applies rate limiting.
│   │   │   ├── auth.controller.ts  Handles HTTP request/response cycle. Calls service. Never touches DB.
│   │   │   ├── auth.service.ts   Business logic: create user, hash password, generate tokens, OTP logic.
│   │   │   ├── auth.validation.ts  Zod schemas for register, login, reset password request bodies.
│   │   │   └── auth.types.ts     TypeScript types specific to auth module.
│   │   │
│   │   ├── user/
│   │   │   ├── user.routes.ts
```

```
|   |   |   ├── user.controller.ts
|   |   |   ├── user.service.ts   Profile read/update, skills, interests, avatar URL generation.
|   |   |   └── user.validation.ts
|   |   |
|   |   ├── discovery/
|   |   |   ├── discovery.routes.ts
|   |   |   ├── discovery.controller.ts
|   |   |   └── discovery.service.ts  Match scoring algorithm. Redis cache read/write. Connections exclusion.
|   |   |
|   |   ├── connections/
|   |   |   ├── connections.routes.ts
|   |   |   ├── connections.controller.ts
|   |   |   └── connections.service.ts  Request/accept/reject/block. Conversation creation on accept.
|   |   |
|   |   ├── chat/
|   |   |   ├── chat.routes.ts    REST endpoints for conversation list, message history, file upload.
|   |   |   ├── chat.controller.ts
|   |   |   └── chat.service.ts   Conversation and message CRUD. Unread count computation.
|   |   |
|   |   ├── portfolio/
|   |   |   ├── portfolio.routes.ts
|   |   |   ├── portfolio.controller.ts
|   |   |   └── portfolio.service.ts  Section updates, publish/unpublish, view tracking enqueue.
|   |   |
|   |   ├── projects/
|   |   |   ├── projects.routes.ts
|   |   |   ├── projects.controller.ts
|   |   |   └── projects.service.ts   CRUD, screenshot presigned URL generation, ownership checks.
|   |   |
|   |   ├── collaboration/
|   |   |   ├── collaboration.routes.ts
|   |   |   ├── collaboration.controller.ts
|   |   |   └── collaboration.service.ts  Post CRUD, applications, saves, expiry logic.
|   |   |
|   |   ├── subscription/
|   |   |   ├── subscription.routes.ts
|   |   |   ├── subscription.controller.ts
```

```
|   |   |   └── subscription.service.ts  Razorpay order creation, HMAC verification, webhook processing.
|   |   |
|   |   ├── github/
|   |   |   ├── github.routes.ts
|   |   |   ├── github.controller.ts
|   |   |   └── github.service.ts  OAuth token storage, GitHub API calls, cache management.
|   |   |
|   |   ├── notifications/
|   |   |   ├── notifications.routes.ts
|   |   |   ├── notifications.controller.ts
|   |   |   └── notifications.service.ts  Create, list, mark-read. Called by other services, not by user directly.
|   |   |
|   |   └── admin/
|   |       ├── admin.routes.ts   Protected by requireAdmin middleware.
|   |       ├── admin.controller.ts
|   |       └── admin.service.ts  User management, report resolution, analytics aggregation.
|   |
|   ├── sockets/                  Socket.IO server logic.
|   |   ├── socket.server.ts      Creates Socket.IO server, attaches Redis adapter, registers namespaces.
|   |   ├── socket.auth.ts        Middleware to validate JWT from handshake.auth.token.
|   |   ├── chat.socket.ts        Handles chat:send, chat:typing, chat:read events.
|   |   └── presence.socket.ts   Handles connect/disconnect, updates lastSeen, broadcasts presence events.
|   |
|   ├── jobs/                     BullMQ queue definitions and worker processes.
|   |   ├── queues.ts             Creates all BullMQ Queue instances. Exported and used by services to add jobs.
|   |   ├── workers.ts            Registers all BullMQ Worker instances. Run as a separate process.
|   |   ├── email.worker.ts       Processes email-queue. Calls AWS SES SDK.
|   |   ├── pdf.worker.ts         Processes pdf-queue. Launches Puppeteer, uploads to S3.
|   |   ├── github.worker.ts      Processes github-sync-queue. Fetches GitHub API data.
|   |   ├── discovery.worker.ts   Processes discovery-queue. Runs MongoDB aggregation, stores in Redis.
|   |   ├── analytics.worker.ts   Processes analytics-queue. Batches portfolio view count updates.
|   |   └── cleanup.worker.ts     Processes cleanup-queue. Daily cron for subscription expiry.
|   |
|   ├── models/                   Mongoose model definitions.
```

		├── User.ts	
		├── Connection.ts	
		├── Conversation.ts	
		├── Message.ts	
		├── Project.ts	
		├── Portfolio.ts	
		├── Subscription.ts	
		├── CollaborationPost.ts	
		├── Notification.ts	
		├── GithubIntegration.ts	
		├── Report.ts	
		└── AuditLog.ts	
	└── utils/	Pure utility functions. No side effects. No Express/Mongoose imports.	
	├── jwt.ts	signAccessToken, signRefreshToken, verifyToken helpers.	
	├── crypto.ts	AES-256 encrypt/decrypt for GitHub tokens.	
	├── paginate.ts	Cursor-based pagination helper for MongoDB queries.	
	├── matchScore.ts	Pure function: computes skill+interest overlap score between two users.	
	└── logger.ts	Winston logger config. JSON in production, pretty-print in dev.	
├── .env		Local dev secrets. Never committed to Git.	
├── .env.example		Template with all variable names but empty values. Committed.	
├── docker-compose.yml		Local dev: MongoDB + Redis containers.	
├── Dockerfile		Production image.	
├── pm2.config.js		PM2 cluster mode config. Runs API + Worker as separate processes.	
├── tsconfig.json			
└── package.json			

11. Frontend — Project Folder Structure

```
frontend/
├── src/
│   ├── app/                                Next.js App Router. Each folder is a route segment.
│   │   ├── layout.tsx                      Root layout. Wraps all pages with Providers (Zustand, TanStack Query, Socket.IO).
│   │   ├── page.tsx                        Landing page. SSG — fully static, fast load, SEO optimised.
│   │   ├── not-found.tsx                   Custom 404 page.
│   │   │
│   │   ├── (auth)/                         Route group — no shared layout with dashboard.
│   │   │   ├── login/page.tsx
│   │   │   ├── register/page.tsx
│   │   │   ├── verify-email/page.tsx       Reads token from URL search params.
│   │   │   ├── forgot-password/page.tsx
│   │   │   └── reset-password/page.tsx
│   │   │
│   │   ├── (dashboard)/                    Route group — shared dashboard layout (sidebar + navbar).
│   │   │   ├── layout.tsx                  Auth guard — redirects to /login if no token.
│   │   │   ├── discover/page.tsx           Discovery feed. Calls GET /discovery/feed.
│   │   │   ├── connections/page.tsx        Accepted connections + pending requests tabs.
│   │   │   ├── chat/
│   │   │   │   ├── page.tsx                Conversation list.
│   │   │   │   └── [conversationId]/page.tsx  Individual chat window.
│   │   │   ├── portfolio/
│   │   │   │   ├── page.tsx                Portfolio builder editor.
│   │   │   │   └── analytics/page.tsx      Portfolio view analytics (Premium).
│   │   │   ├── projects/
│   │   │   │   ├── page.tsx                User's own projects list.
│   │   │   │   ├── new/page.tsx            Create project form.
│   │   │   │   └── [id]/edit/page.tsx      Edit project form.
│   │   │   ├── explore/page.tsx            Public project showcase with filters.
│   │   │   ├── collaboration/
│   │   │   │   ├── page.tsx                Collaboration hub — browse posts.
│   │   │   │   └── new/page.tsx            Create collaboration post (Premium).
│   │   │   ├── notifications/page.tsx      Notification list.
│   │   │   ├── settings/
│   │   │   │   ├── page.tsx                Account settings.
│   │   │   │   ├── subscription/page.tsx   Plan status, upgrade, cancel.
│   │   │   │   └── github/page.tsx         GitHub connect/disconnect.
```

			└── search/page.tsx	User search with filters.
			├── u/[username]/	Public portfolio. SSR with ISR (revalidate: 60s).
			└── page.tsx	Rendered server-side for SEO. Public facing.
			└── admin/	Admin panel. Separate layout with admin nav.
			├── layout.tsx	Admin auth guard — checks role claim in token.
			├── page.tsx	Analytics dashboard.
			├── users/page.tsx	User management table.
			└── reports/page.tsx	Report queue.
		├── components/		Reusable UI components. No business logic.
			├── ui/	Primitive components — used across the entire app.
				├── Button.tsx	Variants: primary, secondary, ghost, danger.
				├── Input.tsx	Controlled input with error state display.
				├── Modal.tsx	Accessible modal with focus trap.
				├── Avatar.tsx	User avatar with fallback initials + online dot.
				├── Badge.tsx	Status/tag badges.
				├── SkeletonCard.tsx	Loading placeholder for cards.
				└── Toast.tsx	Notification toast (uses react-hot-toast).
			├── layout/	Layout chrome components.
				├── Sidebar.tsx	Left nav sidebar. Active state from usePathname.
				├── Navbar.tsx	Top bar — search, notifications bell, user menu.
				└── NotificationBell.tsx	Bell icon with unread count badge. Dropdown list.
			├── user/	Components specific to user/profile domain.
				├── UserCard.tsx	Discovery feed card — avatar, name, skills, connect button.
				├── SkillTag.tsx	Pill tag for a skill.
				├── SkillInput.tsx	Tag input for adding skills with autocomplete.
				└── ReportModal.tsx	Report user modal with reason + description.
			├── chat/	
				├── MessageBubble.tsx	Sent vs received styling, read receipt ticks, timestamps.
				├── TypingIndicator.tsx	Animated three-dot indicator.
				├── MessageInput.tsx	Text input + emoji picker + file attach.

			└── ConversationItem.tsx	Sidebar item — avatar, name, last message, unread dot.
		├── portfolio/		
			├── SectionEditor.tsx	Generic section editor (experience/education list).
			├── ThemePicker.tsx	Portfolio theme selector with live preview.
			└── PortfolioPreview.tsx	Read-only preview panel inside the builder.
		└── projects/		
			├── ProjectCard.tsx	Project thumbnail card.
			└── ScreenshotUploader.tsx	Drag-and-drop multi-file uploader with preview strip.
	├── hooks/			Custom React hooks. Each encapsulates one concern.
		├── useAuth.ts		Reads auth state from Zustand. Returns {user, isLoading, login, logout}.
		├── useSocket.ts		Returns the singleton Socket.IO client. Manages connect/disconnect lifecycle.
		├── usePresence.ts		Subscribes to presence:online/offline events. Returns online user ID set.
		├── useNotifications.ts		Listens to notification:new events. Updates notification count in Zustand.
		├── useInfiniteScroll.ts		IntersectionObserver hook for triggering next page load.
		└── useDebounce.ts		Returns debounced value — used in search inputs.
	├── store/			Zustand stores — global client state.
		├── auth.store.ts		Stores: {user, accessToken}. Actions: setUser, clearAuth.
		├── chat.store.ts		Stores: {conversations, activeConversationId, typingUsers}. Updated by socket events.
		└── notification.store.ts		Stores: {unreadCount, notifications}. Updated by socket events.
	├── lib/			External client configuration and shared utilities.
		├── api.ts		Axios instance. Base URL from env. Interceptor attaches Bearer token from Zustand.
				Interceptor on 401 response: calls /auth/refresh, retries original request.
		├── queryClient.ts		TanStack Query client with default staleTime and retry config.
		└── socket.ts		Creates Socket.IO client singleton. Connects only when user is authenticated.

│ └── types/	TypeScript types. Mirror backend API response shapes.
│ ├── user.types.ts	
│ ├── chat.types.ts	
│ ├── portfolio.types.ts	
│ └── api.types.ts	Generic ApiResponse<T>, PaginatedResponse<T> wrappers.
│	
├── public/	Static assets served directly by Next.js.
│ ├── favicon.ico	
│ └── og-image.png	Open Graph image for social sharing.
│	
├── .env.local	Local dev environment variables. Never committed.
├── .env.example	Template. Committed to Git.
├── next.config.ts	Next.js config: image domains, ISR revalidation, env vars.
├── tailwind.config.ts	Brand colour tokens, font family, custom spacing.
├── tsconfig.json	
└── package.json	

12. Environment Variables (Tentitive)

12.1 Backend (.env)

Variable	Description / Example
NODE_ENV	development | staging | production
PORT	5000
MONGODB_URI	mongodb://localhost:27017/sytu — or Atlas URI in production
REDIS_URL	redis://localhost:6379 — or ElastiCache URL in production
JWT_ACCESS_SECRET	Random 64-char string. Different from refresh secret.
JWT_REFRESH_SECRET	Random 64-char string.
JWT_ACCESS_EXPIRY	15m
JWT_REFRESH_EXPIRY	7d
BCRYPT_ROUND	12
AES_SECRET_KEY	32-char key for GitHub token encryption
AWS_ACCESS_KEY_ID	IAM user key with S3 + SES permissions
AWS_SECRET_ACCESS_KEY	IAM secret
AWS_REGION	ap-south-1 (Mumbai — lowest latency for Indian users)
S3_BUCKET_NAME	sytu-uploads
CLOUDFRONT_URL	https://cdn.sytu.com
RAZORPAY_KEY_ID	rzp_live_xxxxxxxx
RAZORPAY_KEY_SECRET	Never exposed to frontend
RAZORPAY_WEBHOOK_SECRET	Set in Razorpay dashboard — used for HMAC validation
GITHUB_CLIENT_ID	From GitHub OAuth App settings
GITHUB_CLIENT_SECRET	From GitHub OAuth App settings
GITHUB_CALLBACK_URL	https://api.sytu.com/v1/auth/github/callback
FRONTEND_URL	https://sytu.com — for CORS and OAuth redirects
SMS_API_KEY	2Factor.in API key for OTP (free tier: 10k/month)

13. Scalability and Edge Case Handling

13.1 Horizontal Scaling

Component	How It Scales
Node.js API	PM2 cluster mode uses all CPU cores on a single server. Stateless (JWT + Redis) so multiple servers behind Nginx are straightforward.
Socket.IO	Redis pub/sub adapter — all workers share the same channel. A message from a user on Worker 1 reaches a recipient on Worker 4 transparently.
MongoDB	Atlas free tier for development. Atlas M10+ in production supports replica sets for read scaling and automatic failover.
Redis	Single node sufficient at launch. Redis Sentinel for HA in production (3-node, free).
File uploads	Presigned S3 uploads bypass the API entirely. S3 scales infinitely.
Heavy reads (discovery feed)	Precomputed and cached in Redis. Discovery aggregation runs in background, not on request.

13.2 Edge Cases and Handling

Edge Case	How It Is Handled
User closes browser mid-payment	Razorpay sends webhook (payment.captured) to /webhooks/razorpay independently. Backend verifies signature and activates subscription even if browser was closed.
Socket.IO reconnect drops a message	Messages are persisted to MongoDB before the Socket.IO emit. On reconnect, client fetches message history from REST API. No message is lost.
PDF generation times out	BullMQ job has maxAttempts: 3 with exponential backoff. If all fail, job is moved to failed queue and user sees an error state on next poll.
GitHub API rate limit (5000 req/hr)	Sync worker checks X-RateLimit-Remaining header. If < 100, delays sync by calculating reset time and scheduling the BullMQ job accordingly.
Subscription expiry while user is active	isPremium is checked on every premium endpoint via middleware, not only at login. Expiry cron sets it false in MongoDB; middleware re-reads from DB (with Redis cache, 5min TTL).
Duplicate connection request	Unique compound index on {senderId, receiverId} in MongoDB. Duplicate write throws MongoServerError code 11000 — caught in service, returns 409 Conflict.
User deletes account with active subscription	Admin DELETE endpoint cancels Razorpay subscription first, then cascades deletion of all documents. Refund policy is handled out-of-band.
Large file upload (> 10MB)	Multer enforces fileSize limit. Presigned URL has ContentLengthRange condition set. S3 rejects oversized uploads at the edge.

Expired OTP reuse	OTP stored in Redis with TTL: 10min. Key deleted on successful verify. Attempting to verify an expired or used OTP finds no key — returns 400 Invalid or expired OTP.
Portfolio view count race condition	View counts are NOT incremented inline per request. Events are queued in analytics-queue and batch-written via MongoDB $inc every 5 minutes. No race condition possible.
Discovery feed for new user (no skills)	If user has no skills/interests, discovery falls back to a random sample query: db.users.aggregate([{$sample: {size: 20}}]).
WebSocket message to offline user	Server emits to room. If user is offline, room is empty — no error. Notification document is created and email is queued. User sees notification on next login.

14. Deployment Setup

Everything below uses free or low-cost tooling. No Kubernetes or paid orchestration required at launch.

Component	Deployment
Frontend	Vercel (free tier). Connect GitHub repo, auto-deploys on push to main. Handles CDN, edge caching, SSL automatically.
Backend API + Workers	Single AWS EC2 t3.medium (2 vCPU, 4GB RAM). PM2 runs API cluster (cpu_count workers) + Worker process. Cost: ~$30/month.
MongoDB	MongoDB Atlas M0 (free, 512MB) for development. Atlas M10 ($57/month) for production — includes replica set, backups.
Redis	Redis Cloud free tier (30MB) for development. Self-hosted Redis on the same EC2 instance for production (no extra cost).
Reverse Proxy + SSL	Nginx on EC2. Let's Encrypt certificate via Certbot. Free, auto-renews every 90 days.
File Storage	AWS S3 Standard. CloudFront CDN in front. Cost: ~$0.023/GB storage + $0.009/GB transfer.
Email	AWS SES. Free: 62,000 emails/month when sending from EC2.
CI/CD	GitHub Actions. On push to main: run tests, build Docker image, SSH into EC2, pull image, restart PM2.
Domain + DNS	Any registrar (Namecheap, etc.) + Route 53 ($0.50/hosted zone/month) or free DNS via Cloudflare.

14.1 Nginx Config Overview

```
# /etc/nginx/sites-available/sytu
server {
    listen 443 ssl;
    server_name api.sytu.com;

```

```
    ssl_certificate     /etc/letsencrypt/live/api.sytu.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/api.sytu.com/privkey.pem;

    # REST API — round-robin across PM2 workers
    location /v1/ {
        proxy_pass http://localhost:5000;
        proxy_http_version 1.1;
    }

    # Socket.IO — sticky session via IP hash for WS upgrade
    location /socket.io/ {
        proxy_pass http://localhost:5001;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
    }
}

# HTTP to HTTPS redirect
server { listen 80; return 301 https://$host$request_uri; }
```